Cómo funciona el proyecto

La Mesa Perfecta · Web pública, CRM, peticiones a Supabase y Recursos Humanos

Resumen del funcionamiento actual, basado en el código del repositorio. Se explica cómo circulan y se guardan los datos, sin describir el diseño de las pantallas.

1. Arquitectura y conexión

El proyecto utiliza **Next.js y React**. React ejecuta las interacciones del navegador y Next.js también proporciona rutas de servidor. Los datos comerciales se almacenan en **PostgreSQL a través de Supabase**. Recursos Humanos utiliza actualmente archivos del servidor.

El cliente de Supabase se crea en **lib/supabaseClient.ts** con la URL del proyecto y una clave pública configuradas en variables de entorno. La librería convierte instrucciones como **select, insert, upsert, update y delete** en peticiones HTTP a la API de Supabase. Cada respuesta proporciona los resultados en **data** y el posible fallo en **error**.

Recorrido de la petición	Uso en el proyecto
Navegador → Supabase	Formulario público y distintas operaciones de oportunidades, contactos, empresas, tareas y notas.
Navegador → API de Next.js → Supabase	Carga conjunta del directorio comercial mediante /api/crm/directorio.
Navegador → API de Next.js → archivos	Operaciones y documentos de Recursos Humanos.

2. Envío del formulario público

Al pulsar **Continuar o Anterior**, las respuestas permanecen en los campos del navegador. Todavía no se guardan en Supabase. Tampoco hay un borrador persistente que permita recuperarlas después de recargar la página.

Al pulsar **Enviar solicitud**, se validan y recogen todos los campos. Si se ha indicado una empresa, se busca por nombre y se crea si no existe. Después se hace un **upsert en clientes utilizando el email**: se crea el contacto o se actualiza el existente. Con su identificador se inserta una oportunidad en estado **nuevo**, con origen **formulario_web** y los datos del evento.

Mientras se envía se bloquean nuevos envíos. Solo se limpia el formulario cuando la operación confirma el éxito. Si falla, se mantienen las respuestas. Las operaciones son sucesivas, **no una única transacción**: podría guardarse el contacto y fallar después la oportunidad.

Si faltan las columnas adicionales del evento, el código reconoce ese error concreto y conserva sus respuestas en **mensaje**. Si falta clientes.empresa_id, mantiene el nombre en clientes.empresa. La migración **supabase/migracion_formulario.sql** permite añadir las columnas sin borrar los registros existentes.

3. Datos comerciales y relaciones

Tabla	Contenido
empresas	Empresas y sus datos de contacto.
clientes	Personas de contacto y particulares; posibles vínculos con empresas.
usuarios	Comerciales asignables a las oportunidades.
oportunidades	Solicitudes y eventos, presupuesto, fase y responsables.
tareas / notas	Acciones pendientes y anotaciones vinculadas a una oportunidad.
historial_estados	Registro de cambios de fase de las oportunidades.

Una oportunidad se relaciona con un contacto mediante **cliente_id**, con una empresa mediante **empresa_id** y con un comercial mediante **comercial_id**. La consulta OPORTUNIDAD_SELECT recupera la oportunidad junto con estos registros relacionados.

4. Consultas, modificaciones y actualización

La ruta **GET /api/crm/directorio** comprueba la cookie de sesión y consulta empresas, clientes y oportunidades en paralelo. Devuelve un JSON sin caché. El cliente aplica un tiempo límite y puede repetir la lectura si falla. Cuando no hay empresa_id en contactos, intenta deducir la relación a partir del nombre o de sus oportunidades.

La página de oportunidades también consulta directamente Supabase. Cambiar de fase ejecuta un **update filtrado por id**. Cambiar el responsable actualiza comercial_id; cambiar la empresa actualiza empresa_id. Crear tareas o notas utiliza insert y eliminarlas utiliza delete. La aplicación comprueba las respuestas antes de reflejar distintas modificaciones en pantalla.

Los filtros, importes totales y previsiones se calculan principalmente en el navegador con los registros recibidos. **Cierres previstos** selecciona oportunidades abiertas con fecha de evento igual o posterior al día actual y las agrupa por mes; no crea otra tabla de cierres.

Algunas páginas se suscriben a **Supabase Realtime**. Cuando reciben un aviso de cambio vuelven a consultar los datos: el resumen escucha oportunidades y la página de tareas escucha tareas. Esto requiere que Realtime esté habilitado en esas tablas.

El esquema SQL define automatismos para actualizar **updated_at** y registrar cambios de fase en **historial_estados**. Esos automatismos solo funcionan si se han aplicado en la base de datos.

5. Acceso y cierre de sesión

El login envía un **POST a /api/crm/login**. El servidor compara usuario y contraseña con las credenciales configuradas y, si coinciden, crea la cookie **lmp_crm_session**, httpOnly, con una duración de 12 horas. El middleware revisa la cookie al acceder al CRM. Directorio y RRHH la comprueban también en sus rutas de API.

Al confirmar el cierre se envía un **POST a /api/crm/logout**, que elimina la cookie. Si confirma el éxito se vuelve al login; si falla, la aplicación conserva el acceso y muestra el error.

6. Recursos Humanos y documentos

RRHH no guarda actualmente sus datos en Supabase. El navegador consulta GET /api/crm/rrhh y envía las operaciones mediante POST. El servidor comprueba la sesión, valida la acción y actualiza el estado almacenado en archivos.

Operación	Qué hace el servidor
Empleados	Crea o modifica fichas, comprueba el correo y prepara tareas de incorporación.
Fichajes	Registra entradas y salidas; impide una segunda entrada abierta o salir sin haber entrado.
Vacaciones	Valida las fechas, comprueba solapamientos y guarda solicitudes y decisiones.
Selección	Gestiona candidaturas y CV. Al contratar crea una ficha de empleado y sus tareas iniciales.
Evaluaciones	Guarda comentario, trimestre y puntuación de 1 a 5. Impide repetir empleado y trimestre.
Documentos	Recibe fotos, CV y nóminas. Comprueba tipo y tamaño y permite consultar o descargar archivos.

El estado se guarda en **.data/rrhh/state.json**, salvo que HR_DATA_DIR indique otra carpeta. Las fotos y documentos también se guardan en esa carpeta, con identificadores generados. Las subidas utilizan **multipart/form-data**; las demás acciones suelen utilizar JSON.

Los guardados se ordenan en una cola dentro del mismo proceso. Para escribir el estado se crea primero un archivo temporal y después se renombra. Los documentos se sirven desde **/api/crm/rrhh/archivos/[id]**, comprobando la sesión y que el archivo esté registrado.

La conformidad de una nómina genera una copia PDF con los datos de quien firma y la fecha. Es una **firma simulada, sin certificado digital**. El correo corporativo generado al contratar es un dato de la ficha: no crea un buzón real en un proveedor de correo.

7. Qué conviene distinguir del funcionamiento actual

Sesión del CRM y permisos de Supabase son mecanismos distintos. El proyecto no utiliza Supabase Auth para el login. La cookie protege las páginas y ciertas API, pero las peticiones directas a Supabase utilizan la clave pública y se someten a las políticas RLS de la base de datos. El SQL del repositorio propone políticas amplias para la práctica; no equivale a permisos individuales por empleado.

Subir el código a GitHub no modifica las tablas de Supabase. Los scripts SQL deben aplicarse a la base de datos. El despliegue de la web depende del servicio de alojamiento y de su configuración.

La persistencia de RRHH depende del disco del servidor. Si el alojamiento no conserva esos archivos entre ejecuciones o despliegues, los datos no son duraderos. Además, la cola de guardado opera dentro de un proceso; no coordina varias instancias independientes.

Base de la explicación: cliente de Supabase, formulario público, compatibilidad de guardado, middleware, rutas de login/logout, directorio, páginas del CRM, lógica y almacenamiento de RRHH y esquema SQL del repositorio. No contiene contraseñas ni claves.